

Controller

↑

Service

↑

Repository

✓ Day 2 checkpoint

You've now covered:

- JPA vs Hibernate
- @Entity
- @Id
- @GeneratedValue
- Integer vs int
- JpaRepository
- Derived query methods
- @RequestParam
- @PathVariable
- Dependency Injection
- final
- Controller → Service → Repository architecture
- POST/GET/PUT/DELETE
- MySQL verification
- Complete request/response flow

This is a solid Day 2 foundation. Don't add more Spring Boot concepts to this 7–9 AM block yet. The important thing now is being able to explain what you built without looking at the code.

okey so now give me complete notes for the today whatever i learn for interview purpose of springboot

Spring Boot — Day 2 Complete Interview Notes

7 September 2026 | 7:00–9:00 AM Session

Today you built a **Staff REST API with Spring Boot + Spring Data JPA + Hibernate + MySQL** and tested the complete CRUD flow.

These notes are written specifically for **interview revision**. I've kept them aligned with what you studied today.

1. Spring Boot Request Architecture

The main architecture you learned today is:

Client / Postman

↓

Controller

↓

Service

↓

Repository

↓

Spring Data JPA

↓

Hibernate

↓

MySQL

Responsibilities

Layer	Responsibility
Controller	Receives HTTP requests and returns HTTP responses
Service	Handles business/application logic
Repository	Handles database access
JPA	Provides persistence specification
Hibernate	ORM implementation that performs persistence operations
MySQL	Stores the actual data

Interview answer

In a typical Spring Boot application, the Controller handles HTTP requests, the Service layer handles business or application logic, and the

Repository handles database access. Spring Data JPA provides repository functionality, Hibernate performs ORM operations, and MySQL stores the data.

2. JPA

What is JPA?

JPA = Java Persistence API.

JPA is a **specification** that defines how Java objects can be persisted in relational databases.

It provides concepts such as:

@Entity
@Id
@GeneratedValue

Interview answer

JPA is a Java persistence specification that defines rules and interfaces for mapping Java objects to relational database tables and performing persistence operations.

Remember

JPA = Specification / Rules

JPA itself is not the database and is not the ORM implementation.

3. Hibernate

What is Hibernate?

Hibernate is an **ORM framework** and a popular implementation of JPA.

ORM means:

Object Relational Mapping

It maps Java objects to relational database records.

For example:

Java Object



Staff

can be mapped to:

MySQL



staff table

Interview answer

Hibernate is an ORM framework and a popular implementation of JPA. It maps Java objects to database tables and handles database persistence operations.

Easy comparison

JPA → What should be done?

Hibernate → How it is actually done?

4. ORM

What is ORM?

ORM stands for:

Object Relational Mapping

It connects:

Java Object



Database Row

For your project:

Staff Java Object



staff table row

Example:

```
Staff staff = new Staff(  
    null,  
    "Rahul",  
    "Cardiology",  
    "Nurse"  
);
```

can represent:

id	name	department	role
1	Rahul	Cardiology	Nurse

5. @Entity

You used:

```
@Entity  
public class Staff {
```

Purpose

@Entity tells JPA that the class is a **persistent entity** that should be mapped to a database table.

Conceptually:

```
Staff.java  
↓  
@Entity  
↓  
staff table
```

Interview answer

@Entity marks a Java class as a JPA entity so that its objects can be persisted and mapped to a database table.

6. @Id

You used:

```
@Id
private Integer id;
```

Purpose

@Id defines the **primary key** of the entity.

A primary key uniquely identifies each record.

Example:

id	name
1	Rahul
2	Priya
3	Amit

Here id uniquely identifies each staff member.

Interview answer

@Id specifies the primary key of a JPA entity.

7. Why `Integer` Instead of `int`?

This was an important issue you actually encountered today.

Initially:

```
private int id;
```

caused:

Cannot map null into type int

because a primitive int cannot contain null.

You changed it to:

```
private Integer id;
```

Difference

int

↓

primitive
↓
cannot be null

Integer
↓
wrapper class
↓
can be null

For a new Staff object:

id = null

because the database hasn't generated the ID yet.

After saving:

id = 1

Interview answer

We use Integer for a generated entity ID because the ID can be null before the entity is persisted, whereas primitive int cannot contain null.

8. @GeneratedValue

You used:

```
@GeneratedValue(strategy = GenerationType.IDENTITY)  
private Integer id;
```

Purpose

It allows the primary key to be generated automatically according to the configured generation strategy.

In your MySQL setup:

POST request
↓
id = null
↓
Hibernate/JPA

↓
MySQL generates ID
↓
id = 1

You tested this successfully.

You sent:

```
{  
  "name": "Rahul",  
  "department": "Cardiology",  
  "role": "Nurse"  
}
```

and received:

```
{  
  "id": 1,  
  "name": "Rahul",  
  "department": "Cardiology",  
  "role": "Nurse"  
}
```

Interview answer

@GeneratedValue tells JPA that the primary-key value should be generated automatically. With GenerationType.IDENTITY, the database's identity/auto-increment mechanism is used.

9. Entity Class

Your Staff entity conceptually looks like:

```
@Entity  
public class Staff {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Integer id;  
  
    private String name;  
    private String department;  
}
```

```
private String role;

public Staff() {
}
```

Why no-argument constructor?

You had:

```
public Staff() {
}
```

This is important for frameworks such as JPA/Hibernate and for object creation/deserialization.

10. Database

You defined a MySQL database:

```
springboot_training
```

A database is a collection/container used to organize and store data.

Your application connects to:

```
MySQL
↓
springboot_training
```

11. Table

Your staff table stores staff records.

Conceptually:

```
staff
```

```
---
```

```
id | name | department | role
```

1 | Rahul | Cardiology | Nurse

Row

A **row** represents one complete staff record.

1 | Rahul | Cardiology | Nurse

Column

A **column** represents a particular field/attribute.

id
name
department
role

For example:

name column
↓
Rahul
Priya
Amit

12. ArrayList vs MySQL Table

Initially, you used:

```
private final List<Staff> staffList = new ArrayList<>();
```

This stores data in application memory.

For example:

Java Application
↓
ArrayList<Staff>

But after introducing JPA:

Java Application



Repository



Hibernate



MySQL

Why database?

MySQL provides persistent storage and allows database operations to be performed on stored records.

The ArrayList approach is mainly useful for simple in-memory examples/testing, whereas a database is appropriate for persistent application data.

13. JpaRepository

Your repository is conceptually:

```
public interface StaffRepository  
    extends JpaRepository<Staff, Integer> {  
}
```

JpaRepository provides ready-made methods for common persistence operations.

Examples:

```
save()  
findAll()  
findById()  
deleteById()  
existsById()
```

Interview answer

JpaRepository is a Spring Data JPA interface that provides ready-made methods for common database operations, reducing the need to manually implement CRUD operations.

14. Understanding `JpaRepository<Staff, Integer>`

This was another question from today's quiz.

```
JpaRepository<Staff, Integer>
```

means:

`Staff`

↓

Entity type

`Integer`

↓

Primary-key / ID type

Because your entity is:

`Staff`

and its ID is:

`Integer`

we use:

```
JpaRepository<Staff, Integer>
```

Interview answer

In `JpaRepository<Staff, Integer>`, `Staff` represents the entity type and `Integer` represents the type of its primary key.

15. Dependency Injection

You used constructor injection:

```
private final StaffRepository staffRepository;  
  
public StaffService(StaffRepository staffRepository) {  
    this.staffRepository = staffRepository;  
}
```

Spring creates/manages the dependency and provides it to `StaffService`.

Without Dependency Injection

You might think of doing:

```
new StaffRepository()
```

But we don't manually create the Spring-managed repository.

With DI

Spring Container

↓

creates/manages Repository

↓

injects Repository

↓

StaffService

Interview answer

Dependency Injection is a mechanism where Spring manages required objects and injects them into another object instead of the class creating those dependencies itself.

16. Why Constructor Injection?

Your Service:

```
private final StaffRepository staffRepository;  
  
public StaffService(StaffRepository staffRepository) {  
    this.staffRepository = staffRepository;  
}
```

Spring supplies StaffRepository through the constructor.

Benefits you learned today:

- Dependency is explicitly required.
 - The dependency can be stored in a final field.
 - Spring manages the dependency.
 - The class doesn't manually create the dependency.
-

17. Why `final`?

You used:

```
private final StaffRepository staffRepository;
```

`final` means the **reference cannot be reassigned** after initialization.

This is allowed:

```
this.staffRepository = staffRepository;
```

But later:

```
this.staffRepository = anotherRepository;
```

is not allowed.

Interview answer

`final` ensures that the repository reference is initialized once and cannot later be reassigned.

18. Controller

You used:

```
@RestController  
public class StaffController {
```

The Controller handles HTTP requests.

For example:

```
@GetMapping("/api/staff")  
public List<Staff> getStaff() {  
    return staffService.getStaff();  
}
```

The Controller should primarily deal with the **HTTP layer**, while business/application logic belongs in the Service layer.

19. @RestController

@RestController is used for REST API controllers.

It allows controller methods to return data that is written to the HTTP response body.

For example:

```
@GetMapping("/api/staff")
public List<Staff> getStaff() {
    return staffService.getStaff();
}
```

The Java objects are converted into JSON in the HTTP response.

20. @RequestBody

You used:

```
@PostMapping("/api/staff")
public Staff addStaff(@RequestBody Staff staff) {
    return staffService.addStaff(staff);
}
```

When Postman sends:

```
{
  "name": "Rahul",
  "department": "Cardiology",
  "role": "Nurse"
}
```

@RequestBody allows Spring to convert the request JSON into a Staff object.

Conceptually:

```
JSON
↓
@RequestBody
↓
Staff object
```

21. @PathVariable

You used:

```
@GetMapping("/api/staff/{id}")  
public Staff getStaffById(@PathVariable int id) {  
    return staffService.getStaffById(id);  
}
```

Request:

```
GET /api/staff/2
```

The:

```
2
```

is extracted from the URL path.

Interview answer

@PathVariable is used to extract a value from the URL path.

Example:

```
/api/staff/2
```

```
@PathVariable int id
```

22. @RequestParam

You used:

```
@GetMapping("/api/staff/department")  
public List<Staff> getStaffsByDepartment(  
    @RequestParam String department) {
```

Request:

```
GET /api/staff/department?department=Cardiology
```

Here:

```
?department=Cardiology
```

is a query parameter.

Interview answer

@RequestParam is used to extract query parameters from the URL.

23. @PathVariable VS @RequestParam

PathVariable

```
/api/staff/2
```

```
@PathVariable int id
```

RequestParam

```
/api/staff/department?department=Cardiology
```

```
@RequestParam String department
```

Easy memory trick

```
@PathVariable
```

↓

```
/value
```

```
@RequestParam
```

↓

```
?key=value
```

24. Derived Query Method

Your repository contains:

```
List<Staff> findByDepartmentIgnoreCase(String department);
```

This is a **derived query method**.

Spring Data JPA derives the query from the repository method name.

Break it down:

find
↓
By
↓
Department
↓
IgnoreCase

Conceptually:

```
SELECT * FROM staff  
WHERE department = 'Cardiology';
```

The exact SQL generated is handled by the framework.

Interview answer

A derived query method is a Spring Data repository method where the query is derived from the method name.

Examples:

findByName(String name)

findByRole(String role)

findByDepartment(String department)

findByDepartmentIgnoreCase(String department)

25. IgnoreCase

You tested:

Cardiology

and:

cardiology

with:

findByDepartmentIgnoreCase(...)

The method indicates that the comparison should ignore case.

This lets you search without requiring the exact capitalization of the department value.

26. Service Layer

Your Service contains:

```
@Service  
public class StaffService {
```

The Service sits between Controller and Repository.

```
Controller  
  ↓  
Service  
  ↓  
Repository
```

Why?

Because the Controller shouldn't become responsible for all application/business logic.

Interview answer

The Service layer contains application/business logic and acts as an intermediate layer between the Controller and Repository.

27. Why Controller → Service → Repository?

Bad structure:

```
Controller  
  ↓  
Repository
```

This can make the Controller responsible for too much logic.

Better:

Controller



Service



Repository

Each layer has a clear responsibility.

Separation of concerns

Controller → HTTP

Service → Business/application logic

Repository → Database access

This makes the application easier to maintain and extend.

28. save()

You learned:

```
staffRepository.save(staff);
```

save() persists an entity.

For a new entity, it can result in an insert.

Conceptually:

Staff object



save()



Hibernate



SQL INSERT



MySQL

But remember:

save() isn't only for creating data. Depending on the entity state, it can also be used for updates.

29. CRUD Operations

You implemented all major CRUD operations.

C → Create
R → Read
U → Update
D → Delete

Create

POST /api/staff

Read all

GET /api/staff

Read one

GET /api/staff/1

Search

GET /api/staff/department?department=Cardiology

Update

PUT /api/staff/1

Delete

DELETE /api/staff/1

30. POST Request Flow

When you sent:

POST /api/staff

with:

```
{  
  "name": "Rahul",  
  "department": "Cardiology",  
  "role": "Nurse"  
}
```

the flow was:

```
graph TD; Postman --> HTTP_POST[HTTP POST]; HTTP_POST --> Controller; Controller --> RequestBody["@RequestBody"]; RequestBody --> Staff_Java_Object[Staff Java Object]; Staff_Java_Object --> Service; Service --> Repository_save[Repository.save()]; Repository_save --> Spring_Data_JPA[Spring Data JPA]; Spring_Data_JPA --> Hibernate; Hibernate --> SQL; SQL --> MySQL; MySQL --> Record_stored[Record stored];
```

Then:

```
graph TD; MySQL --> Hibernate_JPA[Hibernate/JPA]; Hibernate_JPA --> Repository; Repository --> Service; Service --> Controller; Controller --> Postman;
```

31. GET Request Flow

For:

GET /api/staff

conceptually:

```
Postman
↓
Controller
↓
Service
↓
Repository.findAll()
↓
Spring Data JPA
↓
Hibernate
↓
MySQL
↓
Rows
↓
Hibernate maps rows to Staff objects
↓
Controller
↓
JSON response
↓
Postman
```

32. PUT Request Flow

For:

PUT /api/staff/1

with:

```
{
  "name": "Rahul Sharma",
```

```
"department": "Cardiology",  
"role": "Senior Nurse"  
}
```

the ID comes from:

```
/api/staff/1  
  ↑  
  PathVariable
```

The updated information comes from:

```
@RequestBody
```

Then:

```
Controller  
  ↓  
Service  
  ↓  
Repository  
  ↓  
JPA/Hibernate  
  ↓  
MySQL  
  ↓  
Update
```

33. DELETE Request Flow

For:

```
DELETE /api/staff/1
```

the Controller receives the ID:

```
@PathVariable int id
```

then calls the Service.

The Service uses the Repository/database operation to remove the record.

Conceptually:

DELETE request



Controller



Service



Repository



Hibernate



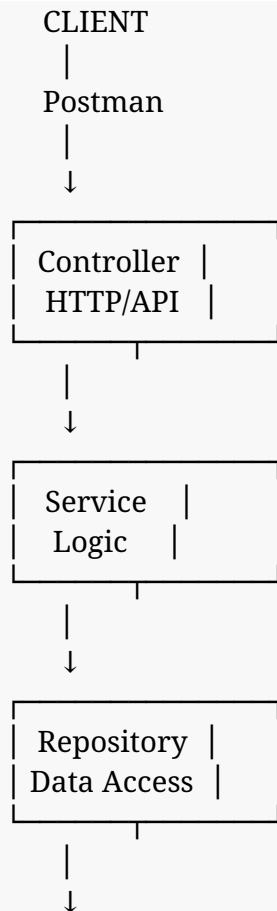
MySQL

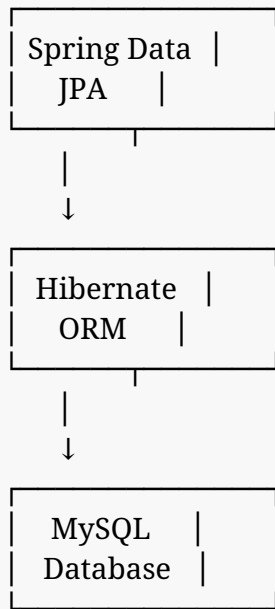


Record deleted

34. Complete Architecture — Memorize This

This is probably the **single most important diagram from today's session**:





35. Most Important Interview Questions From Today

Q1. What is JPA?

JPA is a Java persistence specification that defines rules and interfaces for mapping Java objects to relational databases.

Q2. What is Hibernate?

Hibernate is an ORM framework and a popular implementation of JPA that performs persistence operations.

Q3. JPA vs Hibernate?

JPA → Specification

Hibernate → Implementation / ORM framework

Q4. What is ORM?

Object Relational Mapping maps Java objects to relational database data.

Q5. What does @Entity do?

It marks a class as a JPA entity that can be mapped to a database table.

Q6. What does @Id do?

It defines the primary key of the entity.

Q7. Why use Integer instead of int for generated ID?

Integer can hold null, while int cannot. A newly created entity may have a null ID before the database generates it.

Q8. What does @GeneratedValue do?

It allows the primary key to be generated automatically according to the configured generation strategy.

Q9. What is JpaRepository?

It provides ready-made methods for common persistence operations such as save, findAll, findById, and deleteById.

Q10. What does JpaRepository<Staff, Integer> mean?

Staff is the entity type and Integer is the primary-key type.

Q11. What is Dependency Injection?

Spring manages required dependencies and injects them into the class instead of the class creating them manually.

Q12. Why use final?

It prevents the dependency reference from being reassigned after initialization.

Q13. What does @RequestBody do?

It maps the HTTP request body, such as JSON, to a Java object.

Q14. What is @PathVariable?

It extracts a value from the URL path.

Example:

```
/api/staff/2
```

Q15. What is @RequestParam?

It extracts a query parameter from the URL.

Example:

/api/staff/department?department=Cardiology

Q16. What is a derived query method?

Spring Data JPA derives the database query from the repository method name.

Example:

findByDepartmentIgnoreCase(String department)

Q17. Why use a Service layer?

To separate business/application logic from HTTP handling and database access.

Q18. What does save() do?

It persists an entity and can result in an insert or update depending on the entity state.

Q19. Explain POST request flow.

Postman sends the HTTP request → Controller receives it → @RequestBody converts JSON to a Java object → Service handles the application logic → Repository performs persistence through Spring Data JPA → Hibernate performs ORM/database operations → MySQL stores the data → the response travels back to Postman.

Q20. Why don't we manually write SQL for findByDepartmentIgnoreCase()?

Because Spring Data JPA can derive the query from the repository method name and Hibernate handles the underlying database operation.

36. Your Day 2 One-Minute Revision

Before an interview, remember this:

JPA
= Specification

Hibernate
= ORM + JPA implementation

@Entity
= Entity mapping

@Id
= Primary key

@GeneratedValue
= Automatically generated ID

Integer
= Can be null

JpaRepository
= Ready-made persistence methods

DI
= Spring creates/manages/injects dependencies

Controller
= HTTP layer

Service
= Application/business logic

Repository
= Database access

@RequestBody
= JSON/body → Java object

@PathVariable
= /{value}

@RequestParam
= ?key=value

Derived Query
= Query derived from method name

save()
= Persist entity

CRUD

= Create, Read, Update, Delete

★ Your Day 2 Interview-Level Flow

If an interviewer asks:

“Explain the architecture of your Spring Boot Staff application.”

Give this answer:

“I created a REST API using Spring Boot with a Controller-Service-Repository architecture. The Controller handles HTTP requests, the Service layer handles application logic, and the Repository extends JpaRepository to provide database operations. I used JPA annotations such as @Entity, @Id, and @GeneratedValue to map the Staff entity to MySQL. Spring Data JPA provides the repository abstraction, while Hibernate acts as the ORM implementation and performs the persistence operations against MySQL. I implemented CRUD operations using POST, GET, PUT, and DELETE endpoints and tested them using Postman.”

That is a **strong junior/fresher-level answer** for what you studied today.

✓ Day 2 Status

Spring Boot + JPA + Hibernate + MySQL CRUD: COMPLETE

You didn't just read the concepts today—you **implemented them, fixed a real int/Integer deserialization problem, connected MySQL, and tested the API with Postman**. That's exactly the kind of practical understanding you want for interviews.